

2주차

한 줄 소개

헬스장 등의 장기 이용권 계약 시 사업자 폐업 위험으로부터 소비자를 보호하기 위해, 환불 조건을 스마트 컨트랙트로 블록체인에 고정하는 Web3 시스템

github

<https://github.com/logan0741/blockpass-back.git>

<https://github.com/logan0741/blockpass-AI.git>

프로젝트 정보

- **유형:** 개인/팀 프로젝트
- **기간:** 2주차
- **언어:** Python (FastAPI), JavaScript/TypeScript (React), Solidity
- **인프라:** Docker, Nginx, ngrok, MySQL

사용 기술

기술	용도
React & Vite	계약서 UI, 정책 입력 프론트엔드
FastAPI	DB 관리, 스마트 컨트랙트 컴파일 및 배포 서버
Web3.py	이더리움 네트워크 (EVM) 상호작용, 트랜잭션 관리
Solidity (solc)	환불 정책을 코드로 강제하기 위한 스마트 컨트랙트
MySQL 8.0	오프체인 사용자 데이터 기록
Docker Compose	서비스 컨테이너화 및 통합 실행
ngrok	로컬 서버의 외부 노출 (Webhook/테스트)

시스템 구조도

flowchart TD

```
    subgraph Client["🖥 Frontend (React + Vite)"]
        UI["UI 제공: 계약서 작성 및 서명"] --> JSON_TX["정책 JSON 생성"]
    end

    subgraph Backend["⚙ Backend (FastAPI)"]
        JSON_TX --> API["API 엔드포인트"]
        API --> COMPILER["Solidity 템플릿 코드 주입 및 컴파일 (solc)"]
        COMPILER --> WEB3["Web3.py 트랜잭션 서명"]
        API --> DB["MySQL DB\n(메타데이터)"]
    end

    subgraph Blockchain["🔗 Blockchain (EVM)"]
        WEB3 --> SC["스마트 컨트랙트 배포"]
        SC --> TX_HASH["트랜잭션 해시 반환"]
    end

    TX_HASH --> API
    API --> UI
```

sequenceDiagram

autonumber

actor User as 사업자/사용자

participant FE as Frontend (React)

participant BE as Backend (FastAPI)

participant BLK as Blockchain Network

User->>FE: 환불 정책 (N일 내 M% 환불) 입력

FE->>BE: 정책 데이터 (JSON) 전달

BE->>BE: 정책 -> Solidity 코드 템플릿에 주입 후 컴파일

BE->>BLK: 스마트 컨트랙트 배포 트랜잭션 발생

BLK-->>BE: Contract Address 발급

BE-->>FE: 배포 성공 및 계약 준비 완료

User->>FE: 사용자 계약 확인 및 서명

FE->>BE: 서명 데이터 전송

BE->>BLK: 온체인 계약 기록
BLK-->>FE: 트랜잭션 해시 응답 완료
FE-->>User: 불변의 환불 규정 보장

핵심 구현 요약

1. 자동화된 스마트 컨트랙트 프론트-백 연동

- 일반 사용자가 코드를 몰라도 UI(React)를 통해 **몇 프로, 며칠 내 환불** 같은 정책만 지정하면, 백엔드(FastAPI)가 자동으로 동적 Solidity 코드를 생성 및 컴파일하는 아키텍처 구현.

2. 신뢰 검증 로직 강제화

- '사업자의 양심'에 의존하던 기존 환불 시스템을 탈피하여, 컨트랙트 조건이 충족될 경우 사람의 개입 없이 **블록체인 네트워크가 환불을 강제 실행**하도록 프로세스 설계.

3. 유연한 인프라 구성 (Docker & ngrok)

- `docker-compose` 를 이용해 Frontend, Backend, Nginx를 통합 배포망으로 구축.
- `ngrok` 을 활용한 백엔드 포트 터널링 스크립트(`ngrok_main.py`)를 개발하여 제한된 로컬 환경 및 방화벽 한계를 극복하고 시연 가능한 구조 마련.